

3DAutoMod

Project Team

Ahmad Mujahid	20I-0849
Sakheen Mumtaaz	21I-0275
Nabeel Zaheer	20I-0999

Session 2021-2025

Supervised by

Mr. Irfan Ullah



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

October, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	3D Modeling and Visualization Module	2
1.3.2	User Management Module	2
1.3.3	Mechanic Bidding System Module	3
1.3.4	Communication Module	3
1.3.5	Transaction Management Module	3
1.4	User Classes and Characteristics	4
2	Project Requirements	5
2.1	Use-case/Event Response Table/Storyboarding	5
2.1.1	Use Case Diagram	6
2.1.2	Detailed Use Cases	6
2.1.2.1	User Registration/Login	6
2.1.2.2	Upload Car Image/Video	7
2.1.2.3	Generate Original 3D Model	7
2.1.2.4	Modify Original Images (Car Customization)	8
2.1.2.5	Connect to Car Expert	8
2.1.2.6	Request Bidding Offers	8
2.1.2.7	Facilitate Payment	9
2.1.2.8	Manage All User Accounts	9
2.1.2.9	Manage Payments	10
2.1.3	Event-Response Table	10
2.1.4	Storyboarding	11
2.2	Functional Requirements by Module	11
2.2.1	3D Modeling and Visualization Module	11
2.2.2	User Management Module	11
2.2.3	Mechanic Bidding System Module	12
2.2.4	Communication Module	12
2.2.5	Transaction Management Module	12

2.3	Non-Functional Requirements	12
2.3.1	Reliability	12
2.3.2	Usability	13
2.3.3	Performance	13
2.3.4	Security	13
2.3.5	Portability	13
2.4	Domain Model	14
3	System Overview	15
3.1	Architectural Design	15
3.1.1	Overview	15
3.1.2	Rationale for Choosing MVC	15
3.1.3	Modular Program Structure	16
3.1.3.1	Model	16
3.1.3.2	View	16
3.1.3.3	Controller	16
3.1.4	Relationships Between Modules	17
3.1.5	Architectural Diagrams	17
3.1.5.1	Box and Line Diagram	17
3.1.5.2	Detailed MVC Architecture Diagram	18
3.1.6	Conclusion	18
3.2	Design Models	18
3.2.1	Activity Diagram	20
3.2.2	Data Flow Diagram	21
3.2.3	System Sequence Diagram	22
3.2.4	Entity Relationship Diagram (ERD)	24
3.3	Data Design	24
3.3.1	Information Domain Transformation	25
3.3.2	Data Storage and Organization	25
3.3.3	Data Processing	26
3.3.4	Data Storage Items	26
4		27
4.1	Conclusion	27
	References	29

List of Figures

2.1	Use Case Diagram	6
2.2	Storyboard Representation	11
2.3	Data Representation Diagram (ERD)	14
3.1	Box and Line Diagram of the MVC Architecture	17
3.2	Detailed MVC Architecture Mapping Modules	18
3.3	Process Flow (Activity Diagram)	20
3.4	Class Diagram	21
3.5	user Mechanic inetraction SSD	22
3.6	Car Model Creation SSD	23
3.7	Mechanic Search and Quote Request SSD	23
3.8	Data Representation Diagram (ERD)	24

List of Tables

2.1	Event-Response Table	10
-----	--------------------------------	----

Chapter 1

Introduction

These requirements in this document are to guide the development of 3D AutoMod, an interactive 3D car modification application. This paper is aimed at developers, project managers, testers, and future stakeholders at the automobile industry of customization. The 3D AutoMod project concerns the creation of an entire web application involving a 3D modeling toolkit and a marketplace of automotive enhancements. It comes with an interactive 3D viewer that will allow users to upload images of their vehicles and get 3D models in return. Users shall be able to make changes of these models in real time – changing rims, mirrors, paint jobs or any other aspect. There will be a bidding system where mechanics will be able to bid on modification jobs depending on what the user want.

1.1 Problem Statement

The automotive modifications sector suffers from intense difficulties in converting owners' ideas unto creations. As it is now, enthusiasts do not possess adequate software that would allow them to accurately gauge the car's exterior appearance after applying alterations, which results in time and money being wasted. Furthermore, locating qualified mechanics to accomplish these alterations effectively and cheaply is at times almost inconceivable. They lead to ineffective communication, unfavorable outcomes, and wastage of resources in the automotive customization market. The proposed solution for the above mentioned problems is 3D AutoMod, an AI-supported application that combines a visualization tool with an automotive marketplace to change how automobiles are imagined, designed, and altered.

1.2 Scope

The 3D AutoMod project concerns the creation of an entire web application involving a 3D modeling toolkit and a marketplace of automotive enhancements. It comes with an interactive 3D viewer that will allow users to upload images of their vehicles and get 3D models in return. Users shall be able to make changes of these models in real time – changing rims, mirrors, paint jobs or any other aspect. There will be a bidding system where mechanics will be able to bid on modification jobs depending on what the user want. Explaining the features of the application, there will be an option of live chat to enhance the communication between users and mechanics. A reliable payment gateway will be incorporated for purposes of accomplishing the transactions. Modification history and preferences for the user account will be kept in order to improve the quality for the end user. It will also involve notification on new offers and mechanics responses that should be made to the users of the application. The primary aim of the first few sprints is to deliver a basic, but reliable web application, though the groundwork will be set to make app development possible. It also examines the user management, 3D modeling and visualization, mechanic bidding, communication and transaction management functions which will form the basis for creating an integrated solution for the automotive modification market.

1.3 Modules

The proposed project consists of the following modules:

1.3.1 3D Modeling and Visualization Module

It is responsible for converting uploaded car images to 3D models and allowing real time modification.

1. 3D models generation and segmentation from uploaded images.
2. 3D renderings of the modifications made on cars in real time
3. An interactive modification interface for rims, paint, mirrors, and other non structural features

1.3.2 User Management Module

This module deals with the management of user accounts, login and profile.

1. User registration and login system
2. Profile management with modification history
3. Preference settings and personalization options

1.3.3 Mechanic Bidding System Module

This models the connection between users and mechanics for custom modification work.

1. Mechanic profile creation and management
2. Bidding system for custom modification requests
3. Search and filter functionality for mechanic offers

1.3.4 Communication Module

This module enables direct interaction between users and mechanics.

1. Real-time chat system
2. Notification system for offers and responses
3. Message history and management

1.3.5 Transaction Management Module

This module manages the financial aspects of the platform.

1. Secure payment processing
2. Transaction history and reporting
3. Refund and dispute resolution system

1.4 User Classes and Characteristics

The following user classes are anticipated to use the 3D AutoMod platform:

User class	Description
Car Owners and Enthusiasts	3D AutoMod is targeted at these users, as they are the primary audience to be reached by this product. Individuals who want to make changes to their vehicle before they do it, imagining what it will look like. From the casual car owner to automotive enthusiast there's a wide range. Supposedly they will use the platform very often to explore modification options and interact with mechanics, and they will use the platform to manage their projects.
Automotive Mechanics	Experts who provide modification services through the platform. And they will try to use the bidding system to offer their services, communicate with potential clients and organize their work orders. Their technical skill vary and they may focus on specific type of modifications.
Platform Administrators	Responsible for overall system management of the 3D AutoMod system. User management, content moderation, system maintenance, platform feature operation—all of these will be mundane elements that the team handles.
Automotive Part Retailers	These users may use the platform to showcase their products within the 3D modeling environment, potentially integrating their inventory with the modification options available to car owners.

Chapter 2

Project Requirements

This chapter identifies the project functional and non-functional requirements.

2.1 Use-case/Event Response Table/Storyboarding

Depending on the type of project, the requirement gathering technique or a combination of techniques to be used will be determined. For instance,

- End-user application interactive, such as use case (use case diagram and detailed use case) is rather efficient.
- Event-response table is for real time systems where almost all the functional operations occur at the back end. is ideal for applications where a lot of graphical information is to be displayed.

2.1.1 Use Case Diagram

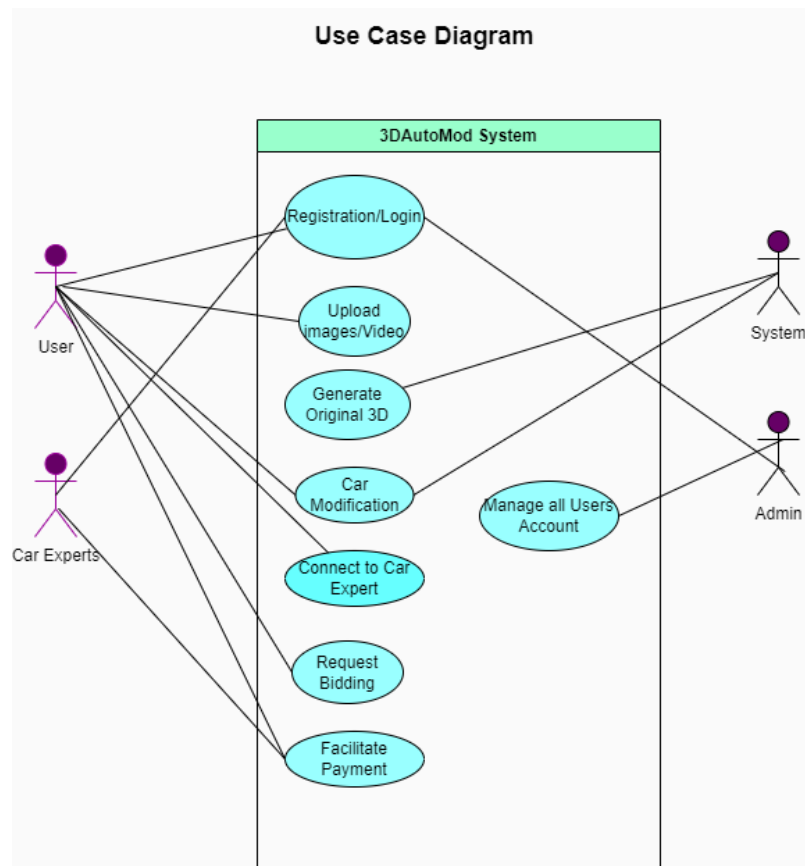


Figure 2.1: Use Case Diagram

2.1.2 Detailed Use Cases

2.1.2.1 User Registration/Login

- **Actors:** User, Admin, Car Customization Experts
- **Description:** Users can create an account or log in to an existing account to access the platform.
- **Precondition:** The user must have a valid email address.
- **Postcondition:** The user gains access to personalized features and saved data.
- **Basic Flow:**
 1. The user navigates to the login/registration page.
 2. The user inputs their email and password.

3. If registering, the user fills in the registration form.
4. The system validates the information and logs in the user or creates a new account.

2.1.2.2 Upload Car Image/Video

- **Actors:** User
- **Description:** Users can upload images or videos of their cars to initiate the customization process.
- **Precondition:** The user must be logged in.
- **Postcondition:** The uploaded media is stored, and the original 3D model generation process is triggered.
- **Basic Flow:**
 1. The user navigates to the upload section.
 2. The user selects an image/video file from their device.
 3. The user submits the file.
 4. The system processes the upload and generates a 3D model.

2.1.2.3 Generate Original 3D Model

- **Actors:** System
- **Description:** The system processes the uploaded media and generates a 3D model of the car.
- **Precondition:** A valid media file must be uploaded.
- **Postcondition:** The original 3D model is available for the user.
- **Basic Flow:**
 1. The system receives the uploaded media.
 2. The system analyzes the media to extract necessary features.
 3. The system generates and stores the 3D model.

2.1.2.4 Modify Original Images (Car Customization)

- **Actors:** User
- **Description:** Users can customize the 3D model through various modification tools (e.g., changing color, adding accessories).
- **Precondition:** The original 3D model must exist.
- **Postcondition:** A modified 3D model is generated and stored.
- **Basic Flow:**
 1. The user accesses the customization tools.
 2. The user applies changes to the 3D model.
 3. The system updates the modified model.
 4. The modified model is saved for future reference.

2.1.2.5 Connect to Car Expert

- **Actors:** User
- **Description:** Users can connect with car experts for advice, bidding offers, and chatting.
- **Precondition:** The user must have a modified 3D model.
- **Postcondition:** The user is connected to a car expert for discussions.
- **Basic Flow:**
 1. The user selects an option to connect with a car expert.
 2. The system displays available experts.
 3. The user chooses an expert to connect with.
 4. The system establishes a chat or discussion thread.

2.1.2.6 Request Bidding Offers

- **Actors:** User
- **Description:** Users can request quotes from car experts for the desired modifications.

- **Precondition:** The user must have a modified 3D model and be connected to an expert.
- **Postcondition:** The car expert sends back bidding offers to the user.
- **Basic Flow:**
 1. The user selects the request bidding option.
 2. The user specifies the desired modifications.
 3. The system forwards the request to the expert.
 4. The expert responds with a bidding offer.

2.1.2.7 Facilitate Payment

- **Actors:** User, Car Expert
- **Description:** Users can make payments through the application after finalizing the deal.
- **Precondition:** Agreement on customization and price.
- **Postcondition:** Payment is processed, and confirmation is sent.
- **Basic Flow:**
 1. The user reviews the final offer.
 2. The user confirms the transaction.
 3. Payment is made through the system.
 4. The system sends a payment confirmation to both user and expert.

2.1.2.8 Manage All User Accounts

- **Actors:** Admin
- **Description:** Admin can view, edit, and delete user accounts as necessary.
- **Precondition:** The admin should be logged in.
- **Postcondition:** User accounts are managed according to admin actions.
- **Basic Flow:**
 1. The admin navigates to the user management section.

2. The admin selects a user account to manage.
3. The admin performs actions (view, edit, delete).
4. The system updates the user account status accordingly.

2.1.2.9 Manage Payments

- **Actors:** Admin
- **Description:** Admin can review and manage payment transactions made through the platform.
- **Precondition:** The admin must be logged in.
- **Postcondition:** Payment records are updated and managed.
- **Basic Flow:**
 1. The admin navigates to the payments section.
 2. The admin reviews transaction records.
 3. The admin can approve or reject transactions.
 4. The system updates the transaction status.

2.1.3 Event-Response Table

The following event-response table outlines the system's responses to specific user actions:

Event	Response
User uploads an image/video	Generate original 3D model
User modifies the 3D model	Update the modified model in real-time
User requests bidding offers	Send requests to connected car experts
User makes a payment	Process payment and send confirmation
Admin logs in	Display user and payment management options

Table 2.1: Event-Response Table

2.1.4 Storyboarding

The storyboard provides a visual representation of the user interaction with the application. It helps outline the user experience and flow through the application. Here, you can describe the storyboard frames, detailing the key interactions and UI elements:

Figure 2.2: Storyboard Representation

Key Frames:

- **Frame 1:** User registration screen where the user inputs email and password.
- **Frame 2:** Upload screen displaying options to upload images/videos.
- **Frame 3:** Customization interface showcasing the 3D model of the car.
- **Frame 4:** Payment confirmation screen after successful payment.

2.2 Functional Requirements by Module

2.2.1 3D Modeling and Visualization Module

- The system shall allow users to upload images or videos of their cars.
- The system shall generate a 3D model from the uploaded media.
- Users shall be able to view and interact with the 3D model in real-time.
- The system shall allow users to modify the 3D model (e.g., change color, add features).
- The system shall store and retrieve car model data for future modifications.

2.2.2 User Management Module

- Users shall be able to register and log in using secure authentication methods.
- Users shall be able to update their profiles and contact information.
- Admins shall be able to manage user accounts, including activating, deactivating, or deleting accounts.
- Admins shall be able to view and manage all user data securely.

2.2.3 Mechanic Bidding System Module

- Users shall be able to send requests for bids to car experts or mechanics.
- Car experts shall be able to submit bids for car customization jobs.
- Users shall be able to accept or reject bids.
- The system shall notify experts when their bid is accepted or rejected.
- Admins shall be able to oversee bidding activities.

2.2.4 Communication Module

- Users shall be able to send messages and chat with car experts.
- Car experts shall be able to reply to user queries.
- The system shall facilitate real-time messaging between users and experts.
- The system shall store conversation history for future reference.

2.2.5 Transaction Management Module

- Users shall be able to view the total cost for accepted bids.
- The system shall allow users to make payments securely through the platform.
- Users shall be able to view their payment history.
- Admins shall be able to track and manage all payment records.
- The system shall generate receipts and confirmations for completed transactions.

2.3 Non-Functional Requirements

The Non-functional requirements for the our system are :

2.3.1 Reliability

- **NFR1:** The system shall maintain an uptime of 99.9%, ensuring reliability and availability for users.
- **NFR2:** The system shall have a mean time between failures (MTBF) of at least 1,000 hours to minimize disruptions.

2.3.2 Usability

- **NFR3:** The user interface shall be intuitive, allowing users to complete common tasks (e.g., uploading media, modifying models) within three clicks.
- **NFR4:** The system shall provide tooltips and help sections to assist users in understanding features and functionalities.

2.3.3 Performance

- **NFR5:** The system shall generate the original 3D model within 5 minutes of media upload.
- **NFR6:** The system shall handle up to 1,000 concurrent users without performance degradation.

2.3.4 Security

- **NFR7:** The system shall implement encryption for sensitive data transmission and storage to protect user information and payment details.
- **NFR8:** The system shall restrict unauthorized access to user accounts and admin functionalities through secure authentication measures.

2.3.5 Portability

- **NFR9:** The system shall be compatible with multiple operating systems (Windows, Linux) to ensure usability across different platforms.
- **NFR10:** The system shall support deployment on various devices (desktops, tablets, smartphones) to accommodate user preferences and accessibility.

2.4 Domain Model

The Domain model below shows the main entities in our car customization platform and their connections. The primary entities involve **User**, and **3D CarModel**, **CustomizationOption**, **Expert**, **Bid**, and **PaymentTransaction**. A user can upload images or videos and get a 3D CarModel based on that, can choose from a number of **CustomizationOptions** and also get in touch with **Experts** to get **Bids** for the customization service. After a bid has been accepted though, the user is in a position to make a **PaymentTransaction** to seal it. This model facilitates proper communication between the users, cars' experts and the system for purposes of customization, bidding, and managing of payments. These entities is the relationships between which define the platform's principles fundamentals based on modularity and scalability.

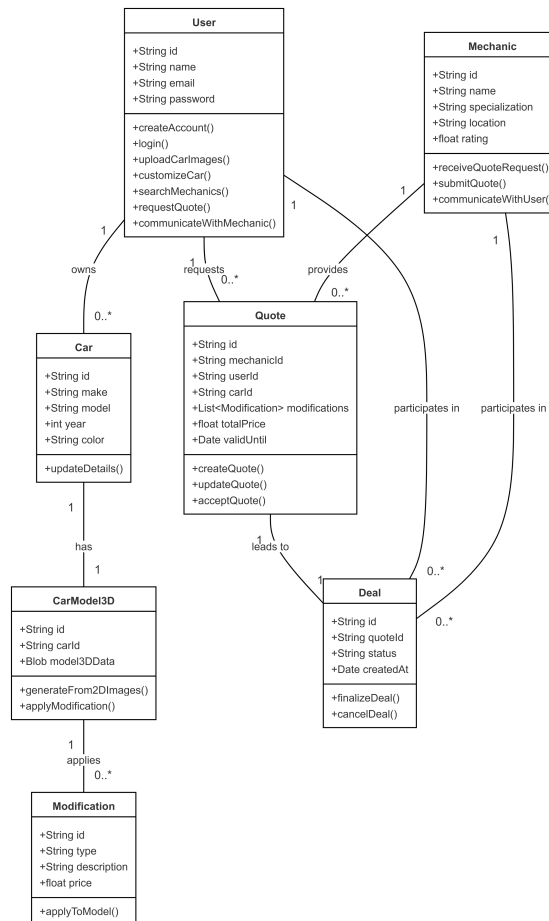


Figure 2.3: Data Representation Diagram (ERD)

Chapter 3

System Overview

We describe project in terms of functionality, context and design in brief.

3.1 Architectural Design

3.1.1 Overview

The structure of the Car Customization Platform is developed using the architectural pattern called **Model-View-Controller (MVC)**. MVC is chosen because it provides clean separation of concerns, improves modularity and promotes easy maintainability. This approach makes it possible to grow, test, and deploy each component of the system without the other components affecting the result.

3.1.2 Rationale for Choosing MVC

- **Separation of Concerns:** MVC distinctly separates data management (Model), user interface (View), and control logic (Controller), enabling organized and manageable code.
- **Modularity:** Each component can be developed and maintained independently, promoting reusability and ease of updates.
- **Scalability:** The architecture supports scaling individual components based on demand without affecting the entire system.
- **Enhanced Collaboration:** Facilitates parallel development, allowing frontend and backend teams to work simultaneously without conflicts.

3.1.3 Modular Program Structure

The system is decomposed into three primary modules corresponding to the MVC components:

3.1.3.1 Model

- **User Model:** Manages user data, including registration, authentication, and profile information.
- **CarModel:** Handles data related to 3D car models, including generation and customization details.
- **ExpertModel:** Manages car expert profiles, bids, and communication logs.
- **PaymentModel:** Oversees payment transactions and records.

3.1.3.2 View

- **Upload Interface:** Provides UI for users to upload images/videos of their cars.
- **3D Model Viewer:** Renders original and customized 3D models using Three.js.
- **Customization Tools:** Interfaces for users to modify 3D models (e.g., color changes, adding accessories).
- **Communication Interface:** Facilitates chat and bidding interactions between users and car experts.
- **Payment Interface:** Presents secure payment forms and transaction statuses.

3.1.3.3 Controller

- **UserController:** Handles user-related operations such as registration, login, and profile updates.
- **MediaController:** Manages the uploading of media and initiates 3D model generation.
- **ModelController:** Oversees the generation and modification of 3D models.
- **ExpertController:** Manages interactions with car experts, including bid submissions and communications.

- **PaymentController:** Facilitates secure payment processing and handles transaction confirmations.

3.1.4 Relationships Between Modules

- **User Interactions:** Users interact with the *Upload Interface* and *3D Model Viewer* (View), which communicate with the *MediaController* and *ModelController* (Controller) to process uploads and modifications.
- **Model Processing:** The *ModelController* interacts with the *CarModel* and *PaymentModel* (Model) to handle business logic and data storage.
- **Expert Communication:** The *Communication Interface* (View) interacts with the *ExpertController* (Controller) to facilitate chats and bidding processes with car experts.
- **Payment Handling:** The *Payment Interface* (View) communicates with the *PaymentController* (Controller) to process transactions securely.
- **Admin Operations:** Admin functionalities interact primarily with the *Model* components to manage user accounts, expert profiles, and monitor system analytics.

3.1.5 Architectural Diagrams

3.1.5.1 Box and Line Diagram

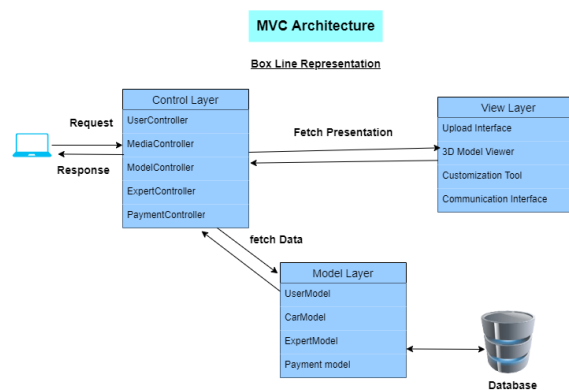


Figure 3.1: Box and Line Diagram of the MVC Architecture

3.1.5.2 Detailed MVC Architecture Diagram

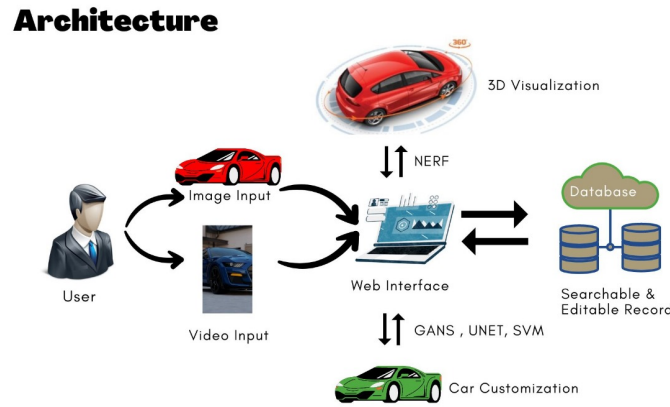


Figure 3.2: Detailed MVC Architecture Mapping Modules

3.1.6 Conclusion

The use of the MVC architectural pattern gives a clear structure and future-proof to the Car Customization Platform. It introduces modularity, modifiability, and expandability of the system, making it withstand demanding tasks like 3D models manipulation, experts engagement, or safe payments.

3.2 Design Models

For our project, we have selected **Object-Oriented Programming (OOP)** as the design methodology. The reasons for choosing OOP include:

- **Modularity:** OOP also allows the construction of a system using objects that represent the real world so as to produce a modular program. This is in harmony with the structure of our platform in which cars, users, experts, and payments are the objects of the model.
- **Reusability:** Code can be used repeatedly in the whole application thus increase in efficiency and reduction in repeated coding. For example, the possibilities of car customization options can be added or changed relatively smoothly to the product without rewriting a large amount of code.

- **Maintainability:** OOP improves the maintainability of the code, it facilitates the modification or upgrading of the system. For instance, adding new levels of customization or new features is possible by means of creating new objects.

Design Models for Object Oriented Development Approach

The object oriented design development may include following diagrams:

- Activity Diagram
- Class Diagram
- Sequence Diagram
- ERD

3.2.1 Activity Diagram

The activity diagram shows how the operations in the system occur, from the time a user uploads an image or video file to the generation of a 3D car model. It also demonstrates the process of customization and how the user engages car specialists for bidding then to secure payment. This diagram presents how the system functions and which process starts which and what processes are initiated as a result.

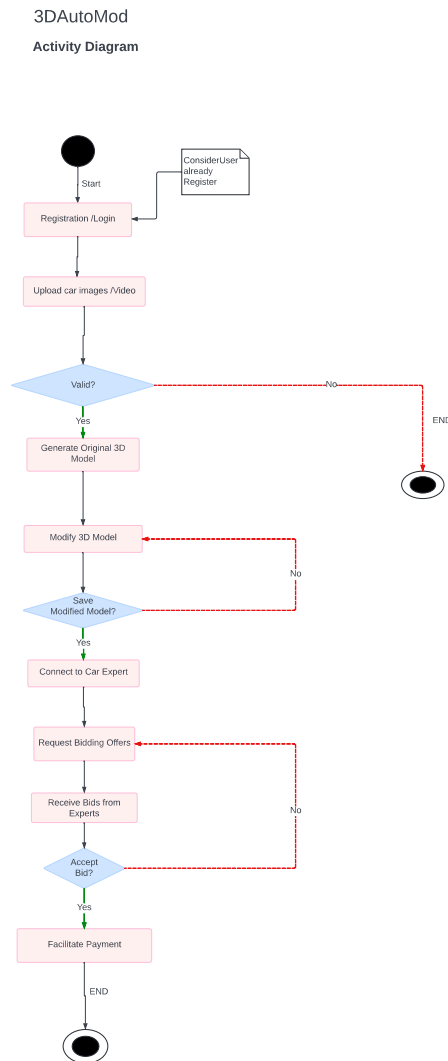


Figure 3.3: Process Flow (Activity Diagram)

3.2.2 Data Flow Diagram

The class diagram shows the static view of the system and displays the classes in the system, their properties and behaviors how they are related. Those are *User* class, *CarModel* class, *Expert* class, and *Payment* class. It also shows site relations betwixt these entities that help in determining the nature of data and procedures within the system. Inter-Object dependencies make reuse and modularity to be coherent and easy to manage hence enhancing the functionality of code.

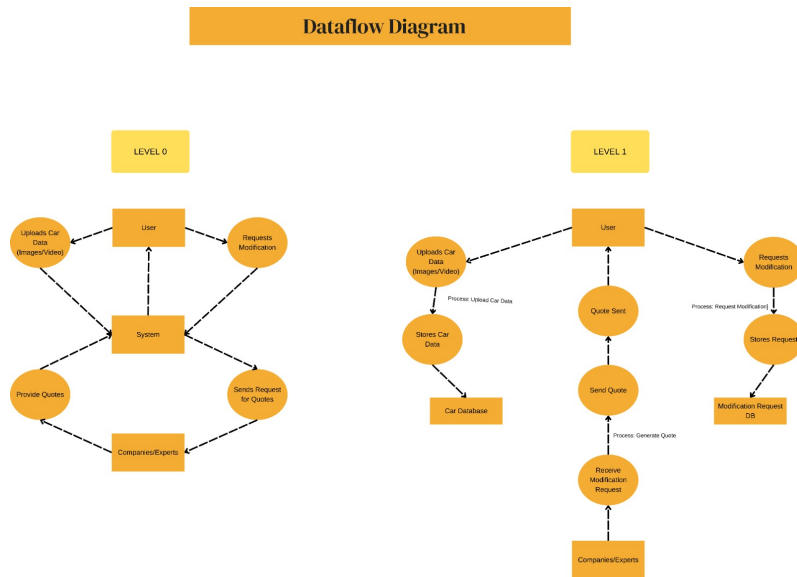


Figure 3.4: Class Diagram

3.2.3 System Sequence Diagram

In sequence diagram, it displays the flow of interaction between different objects in terms of their time axis. For instance, it shows how a user uploads a car image that the system uses to build a 3D model of the car, and apply customizations and interactions with car experts. When using this diagram it becomes easier to follow the sequence of method calls and message passing between the various objects thereby giving a better understanding of the order in which the functionalities are executed.

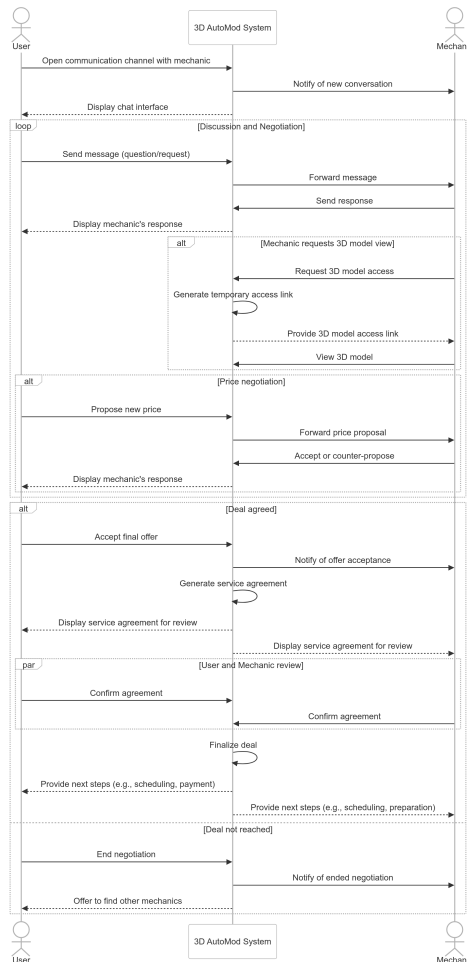


Figure 3.5: user Mechanic interaction SSD

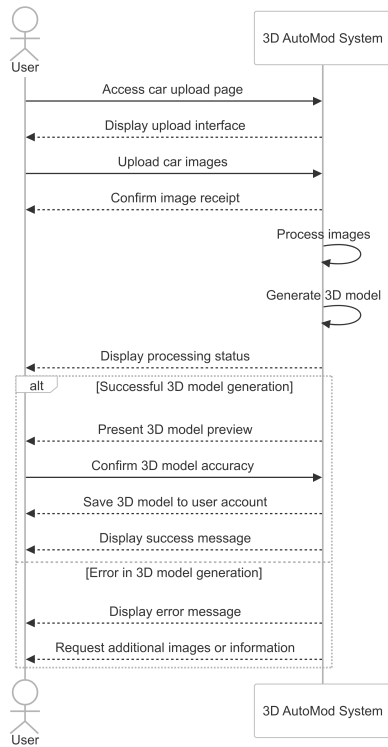


Figure 3.6: Car Model Creation SSD

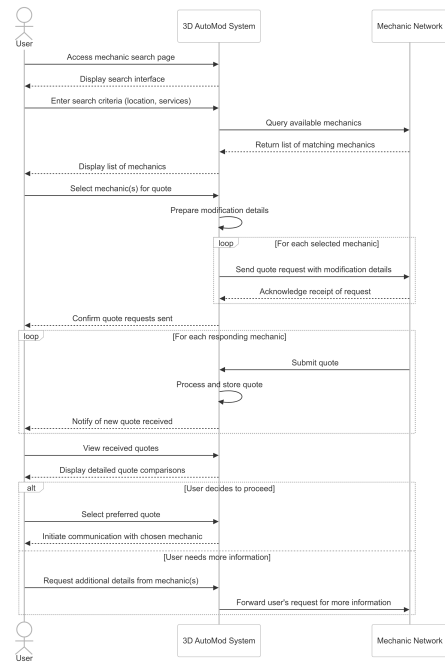


Figure 3.7: Mechanic Search and Quote Request SSD

3.2.4 Entity Relationship Diagram (ERD)

In the ERD, the structure of the data in the system can be communicated, which mainly concerns specifying the association between such entities as User, CarModel, Expert, Payment, etc. It emphasizes one-to-many and many-to-many associations and how data is collected systematically and connected with each other in the database. The ERD assists the identification of relations between data and how data is stored, centralized and can be accessed, stored and managed across various entities due to effective management of the data and consistency.

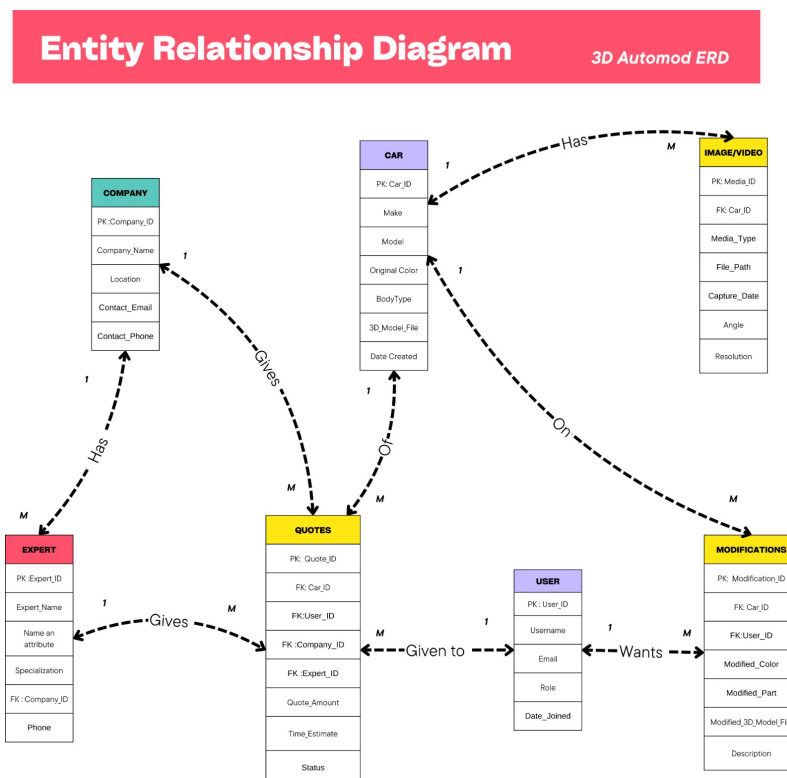


Figure 3.8: Data Representation Diagram (ERD)

3.3 Data Design

The data design for our car customization platform focuses on structuring the information domain into coherent data structures that efficiently store, process, and organize key system entities. Here's how we achieve this:

3.3.1 Information Domain Transformation

The information domain includes such entities as users, cars, possible car tuning, specialists, sales, etc. All of these entities are modeled as structured data to provide better control and search capabilities for the key information. The transformation includes the following steps:

- **Entity Identification:** Each entity within the system is identified based on the application's requirements. Key entities include:
 - User
 - CarModel
 - CustomizationOption
 - Expert
 - PaymentTransaction
- **Attribute Definition:** For each entity, relevant attributes are defined. For example:
 - **User:** user_id, name, email, password, contact_info
 - **CarModel:** car_id, make, model, year, specifications
 - **CustomizationOption:** option_id, car_id, description, price
 - **Expert:** expert_id, name, specialization, rating
 - **PaymentTransaction:** transaction_id, user_id, amount, status, date

3.3.2 Data Storage and Organization

The major data entities are organized into NoSQL structures within MongoDB. Here's how they are processed and organized:

- **MongoDB:** We store our data in MongoDB which is a NoSQL database type. This makes the schema more flexible especially when it comes to the design of the schema, in line with the evolving project. This structure allows a hierarchical representation of the data in the problem domain for document-oriented solutions.
- **Collection Structures:** All the entities are represented by a close MongoDB collection that stores data on entities. This is because in the relationships which exist between the entities, they can either be defined through the use of embedded documents or through references. For instance:

It is assumed that the `users` collection may contain user documents containing embedded arrays for car models. The fact is that the `carModels` collection can contain references to other customization options and therefore, the options of each model can be easily accessed.

- **Document Structure:** Data is stored in JSON like documents which enable a certain flexibility of the fields in the same collection.

3.3.3 Data Processing

Data processing involves various operations such as creating, reading, updating, and deleting (CRUD) records. This is facilitated through a combination of:

- **MongoDB Operations:** MongoDB has a wide range of commands for data manipulation allowing queries and more complex aggregations to be made with little problem.
- **Transaction Management:** MongoDB offers multi-document transaction to cover damage control in the case where several documents might be affected such as payment and modifying users' details.
- **APIs:** A RESTful API will be used for managing the front-end and back-end, so that the data update and exchange could be easily performed.

3.3.4 Data Storage Items

- **MongoDB Database:** The primary repository for all data will be MongoDB database that will include all the required collections and referencing.
- **Cloud Storage:** Users will be able to upload images and videos regarding car customization and those media files will be placed in a cloud storage solution (AWS S3 or Google Cloud Storage) due to large files' size and availability.
- **Cache Storage:** A temporary data storage, where the most frequently accessed data is stored (in-memory caching, such as Redis).

Chapter 4

4.1 Conclusion

In this project, we designed **3Dautomod**, an AI-based system which can reshape the way the users customise their cars. The application lets users upload photographs and videos of their cars, then creates unique 3D models from the data provided, which can be edited using numerous tools. Not only does the system, through the use of the car experts, enable a customizing process in real time but also gives its users professional opinion.

The design of the platform is designed with the principles of usability, reliability and security to allow the users to use the interface effectively. To ensure the user experience is not compromised, as well as ensure protection of users' data, non-functional requirements, including performance criteria and security standards have been incorporated. **3Dautomod** offers a holistic approach that connects car lovers and modders, aspiring and professional, and creates an environment that encourages innovation.

Bibliography